USER INTERFACE DESIGN

EXPERIMENT 3: Difference between CLI(Command line interface) and GUI(Graphical user interface) and VUI (voice userinterface)

Aim:

To design and implement a file renaming system using Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI), and compare their usability and efficiency.

REQUIREMENTS

Hardware:

- Laptop/Desktop
- Microphone

Software:

- Python 3.x
- Tkinter Library
- SpeechRecognition
- PyAudio

Command Line Interface (CLI)

A Command Line Interface allows users to interact with the system by typing commands into a terminal.

Advantages:

- Fast execution
- Low memory usage
- Suitable for advanced users

Disadvantages:

- Requires knowledge of commands
- Not user-friendly for beginners

Implementation:

The CLI program renames files using command-line arguments.

Working:

- Accepts old filename and new filename as inputs
- Uses os.rename() function
- Displays success or error messages

```
import os
import sys

def rename_file(old_name,
  new_name): try:
      os.rename(old_name, new_name)
      print(f"File renamed from {old_name} to
  {new_name}") except FileNotFoundError:
      print(f"Error: {old_name} not
  found.") except Exception as e:
```

```
        print(f"An error occurred: {e}")

if name       == " main
    ": if len(sys.argv) !=
    3:
        print("Usage: python rename_file_cli.py <old_filename> <new_filename>")
    else:
        rename_file(sys.argv[1], sys.argv[2])
```

Graphical User Interface (GUI)

A GUI allows users to interact through graphical elements such as windows, buttons, and text fields.

Advantages:

- Easy to learn
- Interactive
- Reduces user errors

Disadvantages:

- Consumes more system resources
- Slower than CLI

Implementation:

The GUI application is built using Tkinter, allowing users to rename files through a graphical window.

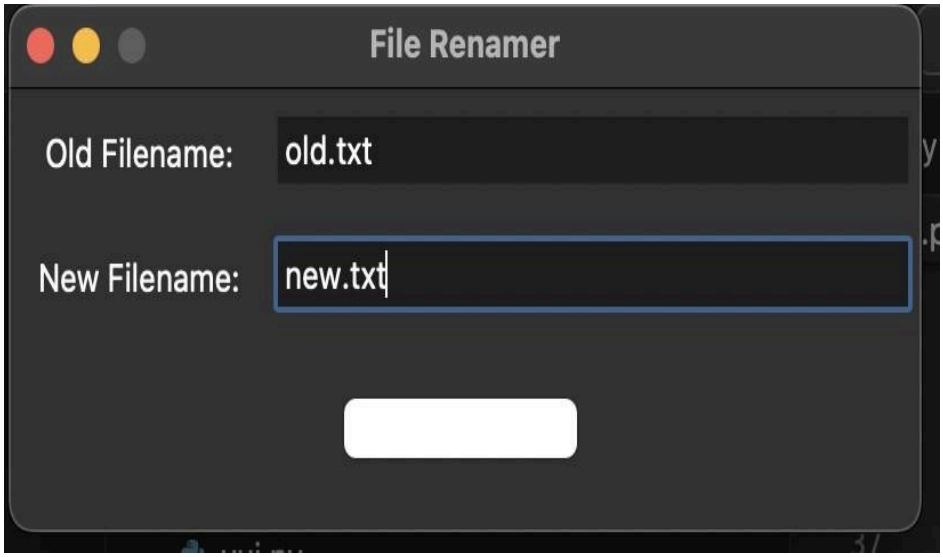
Features:

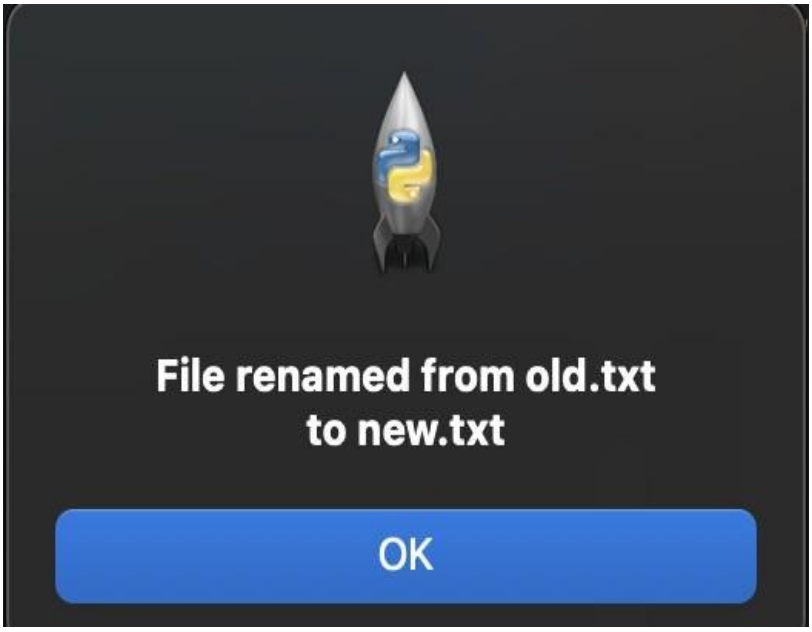
- Input fields for old and new filenames
- Rename button
- Popup messages for success or errors

```
import tkinter as tk
from tkinter import
messagebox import os

def rename_file():
  old_name =
  old_filename_entry.get()
  new_name =
  new_filename_entry.get()

  if not old_name or not new_name:
     messagebox.showwarning("Warning", "Please enter both
     filenames.") return

  try:
     os.rename(old_name, new_name)
     messagebox.showinfo("Success", f"File renamed from {old_name}
  to {new_name}") except FileNotFoundError:
     messagebox.showerror("Error", f"File '{old_name}' not
  found.") except Exception as e:
     messagebox.showerror("Error", f"An error occurred: {e}")

root = tk.Tk()
root.title("File
Renamer")
root.geometry("400x150")   # makes window look
professional root.resizable(False, False)
```

```
new_filename_entry.grid(row=1, column=1)

rename_button = tk.Button(root, text="Rename File", bg="green",
fg="white", command=rename_file)
rename_button.grid(row=2, columnspan=2, pady=15)

root.mainloop()
```

Output:





Voice User Interface (VUI)

A Voice User Interface enables interaction using voice commands and speech recognition technology.

Advantages:

- Hands-free operation
- Improves accessibility
- Natural communication

Disadvantages:

- Dependent on voice clarity
- Sensitive to background noise

Implementation:

The VUI system uses speech recognition to rename files based on voice commands.The spoken command is converted into text, and the file is renamed using Python.

```
import speech_recognition as

sr import os



recognizer = sr.Recognizer()



try:

  with sr.Microphone() as source:

      print("Say: rename old.txt to

      new.txt")

      recognizer.adjust_for_ambient_nois

      e(source) audio =

      recognizer.listen(source)



  try:

      command =

      recognizer.recognize_google(audio)

      print("You said:", command)
```

```
        "pdf": "pdf",
        "doc": "doc"
    }

    # Reconstruct filenames that were split by "dot"
    # Find indices where "dot" appears and merge
    them reconstructed = []
    i = 0
    while i < len(words):
        if i + 2 < len(words) and words[i + 1] == "dot":
            # Merge word + dot + extension
            extension = words[i + 2]
            # Apply corrections to extension
            if extension in corrections:
                extension = corrections[extension]
            reconstructed.append(words[i] + "." + extension)
            i += 3
        else:
            reconstructed.append(words[i]
            ) i += 1

    # Now extract old and new filenames
    # Should be: rename <old_file> to <new_file>
    old_name = reconstructed[1]
```

```
        new_name =
        reconstructed[3]



        os.rename(old_name, new_name)

        print("File renamed
        successfully!")


    except sr.UnknownValueError:

        print("Error: Could not understand

    audio") except sr.RequestError as e:

        print(f"Error: API request failed - {e}")

    except Exception as e:

        print("Error:", e)


except OSError as e:

    print(f"Error: Microphone not available - {e}")

    print("Please ensure a microphone is connected and properly configured.")
```

RESULT:

The file renaming system was successfully implemented using CLI, GUI, and VUI. Each interface allowed users to rename files effectively while demonstrating different levels of usability.